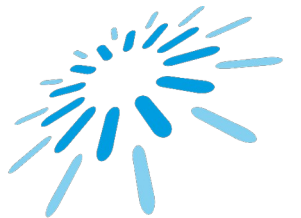




Collaboration, Quality, Test

Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences

About the exam

- Preliminary Date: Wednesday, January 28, 09:00
 - C1 40, Rooms 210, 220, 230
- Duration: 90 Minutes
- You are allowed to bring:
 - Pens, rulers, ...
 - No other materials/devices. Turn your phones off
- No admittance after start of exam

Obvious point about the exam

- Exam is written
- Make sure to write things so that I can read them when grading
 - Illegible answers will be awarded 0 points!

**Recap:
What is Maven Central?**

Learning Objectives for Test Analysis and Design

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **4.1 Test Techniques Overview**
- FL-4.1.1 (K2) Distinguish black-box, white-box and experience-based test techniques
- **4.2 Black-box Test Techniques**
- FL-4.2.1 (K3) Use equivalence partitioning to derive test cases
- FL-4.2.2 (K3) Use boundary value analysis to derive test cases
- FL-4.2.3 (K3) Use decision table testing to derive test cases
- FL-4.2.4 (K3) Use state transition testing to derive test cases
- **4.3 White-box Test Techniques**
- FL-4.3.1 (K2) Explain statement testing
- FL-4.3.2 (K2) Explain branch testing
- FL-4.3.3 (K2) Explain the value of white-box testing

Learning Objectives for Test Analysis and Design

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **4.4 Experience-based Test Techniques**
- FL-4.4.1 (K2) Explain error guessing
- FL-4.4.2 (K2) Explain exploratory testing
- FL-4.4.3 (K2) Explain checklist-based testing
- **4.5. Collaboration-based Test Approaches**
- FL-4.5.1 (K2) Explain how to write user stories in collaboration with developers and business representatives
- FL-4.5.2 (K2) Classify the different options for writing acceptance criteria
- FL-4.5.3 (K3) Use acceptance test-driven development (ATDD) to derive test cases

Chapter 4: Test Analysis and Design



Dynamic Tests: Foundations

Foundations of Black-Box Testing

Equivalence Partitioning

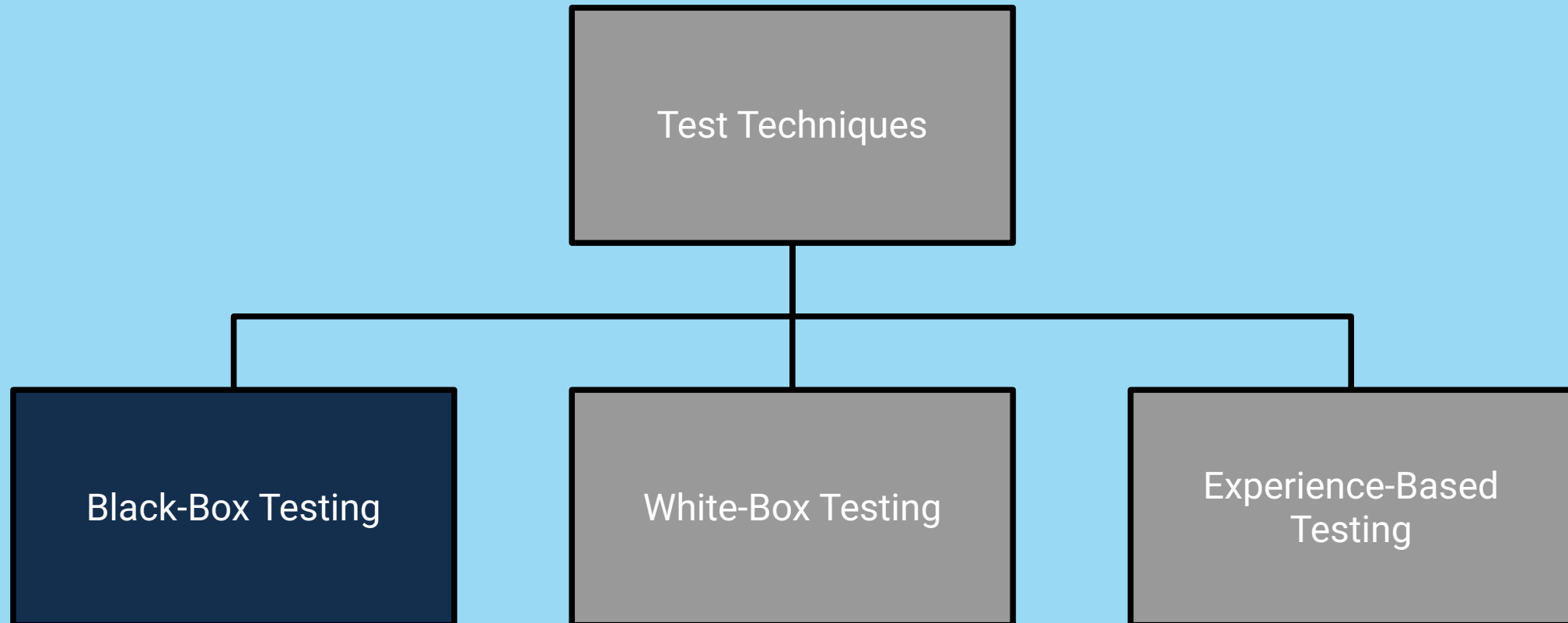
Boundary Value Analysis

Decision Table Testing

...



Test Techniques



Categories of Testing Techniques



Black-box testing

- Test design based on specifications, but **not** on code or inner structure of program
- e.g., equivalence partitioning or boundary value analysis



White-box testing

- Program code is available and basis for test design
- e.g., branch coverage or code coverage

Experience-based testing

- Experience of testers, developers, users, ... about test object and environment
- Experience on where errors occurred in the past or are likely to occur in the future

Chapter 4: Test Analysis and Design

Dynamic Tests: Foundations



Foundations of Black-Box Testing

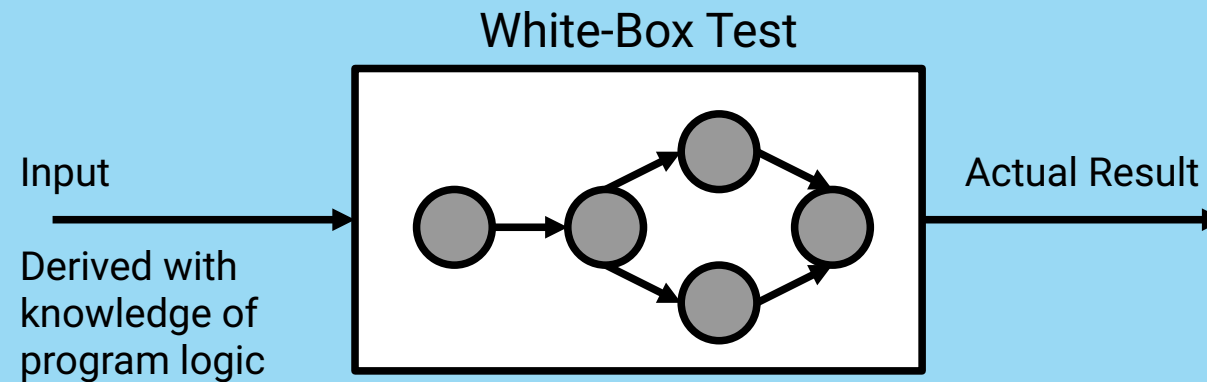
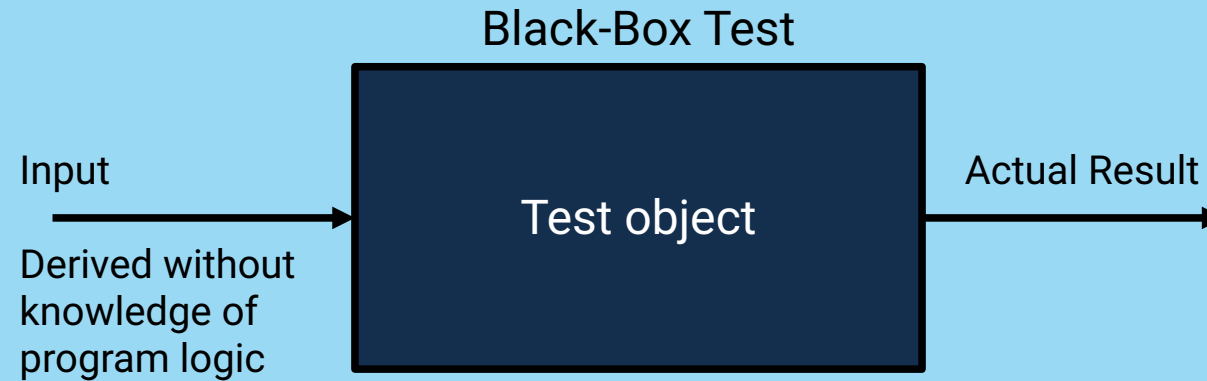
Equivalence Partitioning

Boundary Value Analysis

Decision Table Testing

...

Black-Box Testing vs. White-Box Testing (1/2)

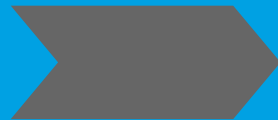


What are advantages / disadvantages of each testing technique?

Chapter 4: Test Analysis and Design

Dynamic Tests: Foundations

Foundations of Black-Box Testing



Equivalence Partitioning

Boundary Value Analysis

Decision Table Testing

...

Learning Objectives for Test Analysis and Design

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **4.1 Test Techniques Overview**
- FL-4.1.1 (K2) Distinguish black-box, white-box and experience-based test techniques
- **4.2 Black-box Test Techniques**
- FL-4.2.1 (K3) Use equivalence partitioning to derive test cases
- FL-4.2.2 (K3) Use boundary value analysis to derive test cases
- FL-4.2.3 (K3) Use decision table testing to derive test cases
- FL-4.2.4 (K3) Use state transition testing to derive test cases
- **4.3 White-box Test Techniques**
- FL-4.3.1 (K2) Explain statement testing
- FL-4.3.2 (K2) Explain branch testing
- FL-4.3.3 (K2) Explain the value of white-box testing

Basic Idea of Equivalence Partitioning (1/3)

Age must be ≥ 18 and less than 130, otherwise error notification

Age

What values would you test with?

Basic Idea of Equivalence Partitioning (2/3)

Age must be ≥ 18 and less than 130, otherwise error notification

Age

Define Equivalence Partitions

EP1 $< 18 \rightarrow$ Invalid Input

EP2 ≥ 18 and $< 130 \rightarrow$ Valid Input

EP3 $> 130 \rightarrow$ Invalid Input

Basic Idea: All values in a partition trigger **equivalent behavior** of test object

Basic Idea of Equivalence Partitioning (3/3)

Age must be ≥ 18 and less than 130, otherwise error notification

Age

Define Equivalence Partitions

EP1 $< 18 \rightarrow$ Invalid Input $\rightarrow 5$

EP2 ≥ 18 and $< 130 \rightarrow$ Valid Input $\rightarrow 90$

EP3 $> 130 \rightarrow$ Invalid Input $\rightarrow 10000$

Choose one representative each

Basic Idea: All values in a partition trigger equivalent behavior of test object

Equivalence Partitioning – Basic Idea

- 1) Divide value ranges of inputs/outputs into equivalence partitions (EP)
- 2) All values of a partition show equivalent behavior of the test object
- 3) Meaningful sample: only one test value per EP (representative)
 - If one value of the EP reveals an error, it is hoped that every other value of the EC will also reveal this error
 - If a value of the EP does not detect an error, it is hoped that no other value of the EC will detect an error either
- Make sure that equivalence classes do not overlap!

Equivalence Partitioning

A black-box test technique in which the test cases are designed with regard to the execution of equivalence partitions, whereby one representative of each equivalence class is used. [After ISO 29119]

Equivalence Partitioning – Examples (1/12)

- Boundary specifies **value range**:
 - One valid EP
 - Two invalid EPs

Example Specification: integer inputs between 1 and 100 are allowed:

- Value range of input: Integer numbers
- Valid equivalence partition: $1 < x < 100$
- Invalid equivalence partitions: $x \leq 1$ and $x \geq 100$

Equivalence Partitioning – Examples (2/12)

- No specified **numeric value** boundary:
 - One valid EP
 - Two invalid EPs

Example Specification: inputs of type integer:

- Value range of input: Integer numbers
- Valid equivalence partition: [MIN_INT MAX_INT]
- Invalid equivalence partitions: MIN_INT - 1, MAX_INT + 1
- Invalid values are numbers greater than or smaller than interval bounds or **all non-numeric values or floating point number, etc.** → Robustness test

Equivalence Partitioning – Examples (3/12)

- No specified **numeric value** boundary
- Choose representatives:

Example Specification: inputs of type integer:

- Valid equivalence partition: [MIN_INT MAX_INT]
→ negative number, positive number, 0
- Invalid equivalence partitions: MIN_INT - 1, MAX_INT + 1
→ Number < MIN_INT, Number > MAX_INT
- Generally, a single value could be used per EP. Due to heuristics a negative value, a positive value, and 0 are commonly chosen.
- An expected output must be specified per input

Equivalence Partitioning – Examples (4/12)

- Boundary specifies **number of values**:
 - One valid EP
 - Two invalid EPs

Example Specification: Each member of a sports club must specify at least one sport they are involved in. Each member can actively participate in up to three sports.

- Valid equivalence partition: $1 \leq x \leq 3$
- Invalid equivalence partitions: $x = 0$ (none), $x > 3$ (too many)

Equivalence Partitioning – Examples (5/12)

- Boundary specifies **set of values**:
 - Separate valid EP for each value in set
 - One total invalid EP

Example Specification: Our sports club offers the following: Soccer, Hockey, Handball, Basketball, and Volleyball.

- Valid equivalence partitions: 1 to 3 sports of: Soccer, Hockey, Handball, Basketball, Volleyball
- Invalid equivalence partition: anything else, such as Badminton

Equivalence Partitioning – Examples (6/12)

- Boundary specifies **situation that must be met**:
 - One valid EP
 - One invalid EP

Example Specification: Each sports club member has a membership number. The number starts with the first letter of the family name.

- Valid equivalence partition: First character is a letter
- Invalid equivalence partition: First character is not a letter (e.g., a digit or special character)

Falsehoods programmers believe about names

(mostly U.S.)

- 1) People have exactly one canonical full name.
- 2) People have exactly one full name which they go by.
- 3) People have, at this point in time, exactly one canonical full name.
- 4) People have, at this point in time, one full name which they go by.
- 5) People have exactly N names, for any value of N.
- 6) People's names fit within a certain defined amount of space.
- 7) People's names do not change.
- 8) People's names change, but only at a certain enumerated set of events.
- 9) People's names are written in ASCII.
- 10) People's names are written in any single character set.
- 11) People's names are all mapped in Unicode code points.
- 12) People's names are case sensitive.
- 13) People's names are case insensitive.
- 14) People's names sometimes have prefixes or suffixes, but you can safely ignore those.
- 15) People's names do not contain numbers.
- 16) People's names are not written in ALL CAPS.
- 17) People's names are not written in all lower case letters.
- 18) People's names have an order to them. Picking any ordering scheme will automatically result in consistent ordering among all systems, as long as both use the same ordering scheme for the same name.
- 19) People's first names and last names are, by necessity, different.
- 20) People have last names, family names, or anything else which is shared by folks recognized as their relatives
- 21) People's names are globally unique
- 22) People's names are almost globally unique.
- 23) Alright alright but surely people's names are diverse enough such that no million people share the same name.
- 24) My system will never have to deal with names from China
- 25) Or Japan.
- 26) Or Korea.
- 27) Or Ireland, the United Kingdom, the United States, Spain, Mexico, Brazil, Peru, Russia, Sweden, Botswana, South Africa, Trinidad, Haiti, France, or the Klingon Empire, all of which have "weird" naming schemes in common use.
- 28) That Klingon Empire thing was a joke, right?
- 29) Confound your cultural relativism! People in my society, at least, agree on one commonly accepted standard for names.
- 30) There exists an algorithm which transforms names and can be reversed losslessly. (Yes, yes, you can do it if your algorithm returns the input. You get a gold star.)
- 31) I can safely assume that this dictionary of bad words contains no people's names in it.
- 32) People's names are assigned at birth.
- 33) OK, maybe not at birth, but at least pretty close to birth.
- 34) Alright, alright, within a year or so of birth.
- 35) Five years?
- 36) You're kidding me, right?
- 37) Two different systems containing data about the same person will use the same name for that person.
- 38) Two different data entry operators, given a person's name, will by necessity enter bitwise equivalent strings on any single system, if the system is well-designed.
- 39) People whose names break my system are weird outliers. They should have had solid, acceptable names, like 田中太郎
- 40) People have names.



Equivalence Partitioning – Examples (7/12)

☒ Passcode

Only users who have the invite link or passcode can join the meeting

☐ Waiting Room

Only users who are in the waiting room can join the meeting

Passcode must:

✓ Have at least 7 characters

What are valid and invalid
Equivalence Partitions
regarding passcode
length?

Equivalence Partitioning – Examples (8/12)

☒ Passcode

Only users who have the invite link and passcode can join the meeting

☐ Waiting Room

Only users already in the meeting

Passcode must:

- ✓ Have at least 7 characters

What are valid and invalid Equivalence Partitions regarding passcode length?

- Valid Equivalence Partition:
 - EQ1: Passcode has 7 or more characters
- Invalid Equivalence Partition:
 - EQ2: Password has fewer than 7 characters
 - EQ3: Password has more than MAX_INT characters

Equivalence Partitioning – Examples (9/12)

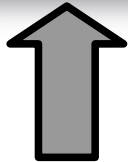
When

Duration hr min

Time Zone

☐ Recurring meeting

AM
PM



What are valid and invalid Equivalence Partitions?

- Valid Equivalence Partition:
 - EQ1: Valid Selection of AM / PM
- Invalid Equivalence Partition:
 - EQ2: Anything else except AM / PM

Equivalence Partitioning – Examples (10/12)

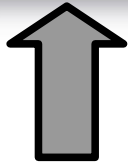
When

Duration hr min

Time Zone

☐ Recurring meeting

AM
PM



Note on Testability:
Don't give your users the option of entering wrong data!

- Valid Equivalence Partition:
 - EQ1: Valid Selection of AM / PM
- Invalid Equivalence Partition:
 - EQ2: Anything else except AM / PM

Equivalence Partitioning – Examples (11/12)

When

PM

AM

PM

Duration hr min

Time Zone

☐ Recurring meeting



What are valid and invalid
Equivalence Partitions?

Equivalence Partitioning – Examples (12/12)

Schedule Meeting

Topic

My Meeting



What are valid and invalid
Equivalence Partitions?

Defining Test Cases

- To specify a test case, an input value must be assigned to each parameter
- To do this, a decision must be made as to which of the available representatives are to be combined to form an input data set

The image shows a meeting creation interface with the following fields and options:

- Topic:** My Meeting
- + Add Description**
- When:** 11/12/2024, 1:00, PM
- Duration:** 1 hr, 0 min
- Time Zone:** (GMT+1:00) Amsterdam, Berlin, Rome, Stockholm, Vienna
- ☐ Recurring meeting
- Attendees:** Enter user names or email addresses
- Registration:** ☐ Required
- Meeting ID:** ☒ Generate Automatically, ☐ Personal Meeting ID 626 610 9132
- Template:** Select a template
- Whiteboard:** ☐ Add Whiteboard
- Security:**
 - ☒ Passcode: 3688222
 - Only users who have the invite link or passcode can join the meeting
 - ☒ Waiting Room
 - Only users admitted by the host can join the meeting
 - ☐ Require authentication to join
- Video:**
 - Host: ☐ on, ☒ off
 - Participant: ☐ on, ☒ off
- Audio:**
 - ☐ Telephone, ☐ Computer Audio, ☒ Both
 - Dial from Germany
- Options:** Show

Question

**That's a lot of possible combinations!
What do we do?**

Topic

[+ Add Description](#)

When

Duration hr min

Time Zone

☐ Recurring meeting

Attendees

Registration ☐ Required

Meeting ID ☒ Generate Automatically ☐ Personal Meeting ID 626 610 9132

Template

Whiteboard

Security ☒ Passcode
Only users who have the invite link or passcode can join the meeting

☒ Waiting Room
Only users admitted by the host can join the meeting

☐ Require authentication to join

Video Host ☐ on ☒ off
Participant ☐ on ☒ off

Audio ☐ Telephone ☐ Computer Audio ☒ Both
Dial from Germany

Options [Show](#)

Minimizing Number of Test Cases – Heuristics

- At minimum: At least one representative of each equivalence class in at least one test case!
- Combine test cases from all representatives and then sort by frequency (**usage profiles**)
 - Then prioritize test cases in this order
 - **Only test with usage-relevant test cases**
 - Give preference to test cases that contain limit values or combinations thereof
- Test each representative of an equivalence class with each representative of every other equivalence class in combination
 - i.e. **pairwise combination** instead of complete combination
- Do not combine representatives of invalid equivalence classes with representatives of other invalid equivalence classes!
 - Maximum of one invalid equivalence class per test case; **error masking possible**

Equivalence Partitioning – End Criterion

- End Criterion for tests when using Equivalence Partitions

$$\text{EP Coverage} = \text{Number of tested EPs} / \text{Total EPs}$$

- Example:
 - Defined end criterion: EP coverage at least 80%
 - 18 partitions determined from the test basis
 - 15 EPs tested
 - EP coverage = $15 / 18 \approx 83.33 \%$
 - End criterion fulfilled!

Equivalence Partitioning – Advantages and Disadvantages

Advantages

- Number of test cases smaller compared to unsystematic debugging
- Suitable for programs with many different input and output conditions

Disadvantages

- Considers conditions for individual input or output parameters
- Consideration of interactions and dependencies of conditions very time-consuming

Recommendation

- To select effective test data:
Combination of EPs with error-oriented methods, e.g. boundary value analysis

Chapter 4: Test Analysis and Design

Dynamic Tests: Foundations

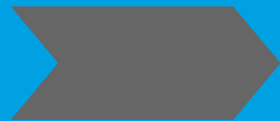
Foundations of Black-Box Testing

Equivalence Partitioning

Boundary Value Analysis

Decision Table Testing

...



Learning Objectives for Test Analysis and Design

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **4.1 Test Techniques Overview**
- FL-4.1.1 (K2) Distinguish black-box, white-box and experience-based test techniques
- **4.2 Black-box Test Techniques**
- FL-4.2.1 (K3) Use equivalence partitioning to derive test cases
- FL-4.2.2 (K3) Use boundary value analysis to derive test cases
- FL-4.2.3 (K3) Use decision table testing to derive test cases
- FL-4.2.4 (K3) Use state transition testing to derive test cases
- **4.3 White-box Test Techniques**
- FL-4.3.1 (K2) Explain statement testing
- FL-4.3.2 (K2) Explain branch testing
- FL-4.3.3 (K2) Explain the value of white-box testing

Boundary Value Analysis – Basic Idea (1/2)

Age must be ≥ 18 and less than 130, otherwise error notification

Age

Define Equivalence Partitions

EP1 $< 18 \rightarrow$ Invalid Input $\rightarrow 5$

EP2 ≥ 18 and $< 130 \rightarrow$ Valid Input $\rightarrow 90$

EP3 $> 130 \rightarrow$ Invalid Input $\rightarrow 10000$

Choose one representative each

Basic Idea: Branches and loops tend to have “boundary values” that “barely/just” fulfill or do not fulfill a condition.

Boundary Value Analysis – Basic Idea (2/2)

Age must be ≥ 18 and less than 130, otherwise error notification

Age

Define Equivalence Partitions

EP1 $< 18 \rightarrow$ Invalid Input $\rightarrow 5 \rightarrow 17$

EP2 ≥ 18 and $< 130 \rightarrow$ Valid Input $\rightarrow 90 \rightarrow 18, 129$

EP3 $> 130 \rightarrow$ Invalid Input $\rightarrow 10000 \rightarrow 130$

Choose one representative each

Basic Idea: Branches and loops tend to have “boundary values” that “barely/just” fulfill or do not fulfill a condition.

Boundary Value Analysis (1/3)

- Idea: Branching and loop conditions often feature boundary values for which the condition just still applies (or just no longer applies)
 - These cases are error-prone (off by one)
 - Test data checking such boundary values are more likely to detect failures than test data that do not
- Best results when combined with other methods
- When combined with equivalence partitioning:
 - Test the limits of the EP: smallest and largest value per EP
 - Each boundary of an EP must occur in a test data combination

Boundary Value Analysis

A black-box test technique in which test cases are designed based on boundary values.

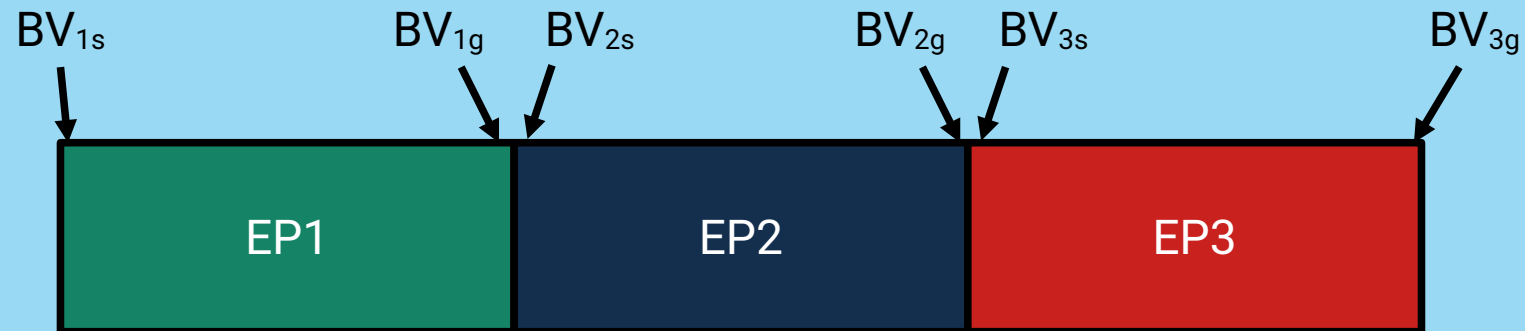
[After ISO 29119, Glossary ITSQB V3.3]

Boundary Value Analysis (2/3)

- Boundary values of valid EP: “valid” boundary values
- Boundary values of invalid EP: “invalid” boundary values
- Boundary values of the EP can be supplemented by further values to be tested, which increase the probability of detecting a failure.
 - Values to the right or left of the boundaries (invalid values, smaller or larger than the limit)
- Quantitative ranges (e.g. for data structures, relationships)
 - Smallest and largest valid number
 - Smallest and largest invalid number

Boundary Value Analysis (3/3)

At minimum: Test the smallest and greatest values of each EP.
Adding additional values increases the probability of uncovering failures.



BV_{1s} – Smalles Boundary Value of EP 1

BV_{1g} – Greatest Boundary Value of EP 2

Boundary Value Analysis – End Criterion

- End Criterion for tests when using Boundary Value Analysis

$$\text{BV Coverage} = \text{Number of tested BVs} / \text{Total BVs}$$

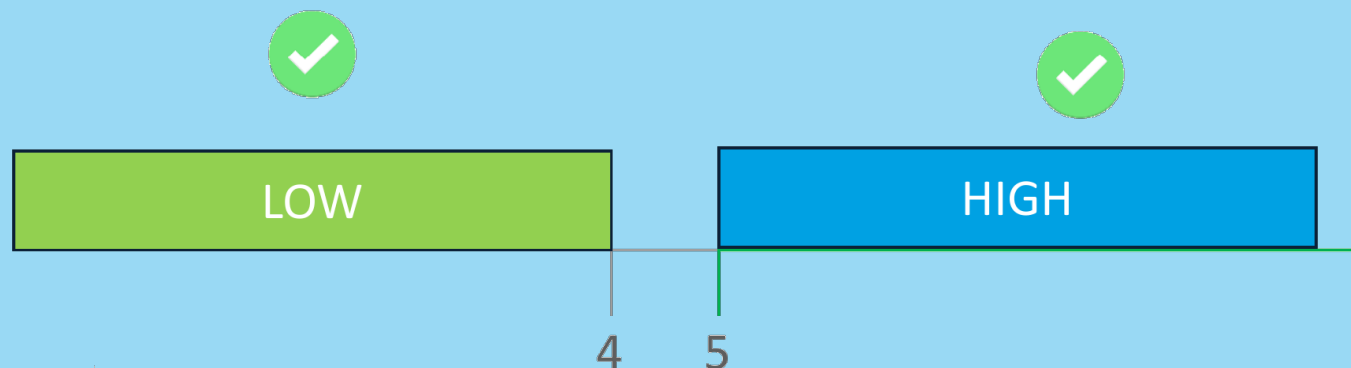
- Example:
 - Defined end criterion: BV coverage at least 90%
 - 20 boundary values determined from the test basis
 - 19 tested
 - BV coverage = $19 / 20 \approx 95 \%$
 - End criterion fulfilled!

Boundary Value Analysis – Supplementary (1/4)

- Basic question: How intensively is the boundary area tested?
- In some cases it is worth adding further values close to the boundary
- Example:
 - Condition " $x \geq 5$ ": If $x \geq 5$, HIGH should be output otherwise LOW
 - Boundary values: $x = 5$ (HIGH) and $x = 4$ (LOW)

PSEUDO-CODE

```
if (x <= 5) {  
    print („HIGH“);  
} else {  
    print („LOW“);  
}
```

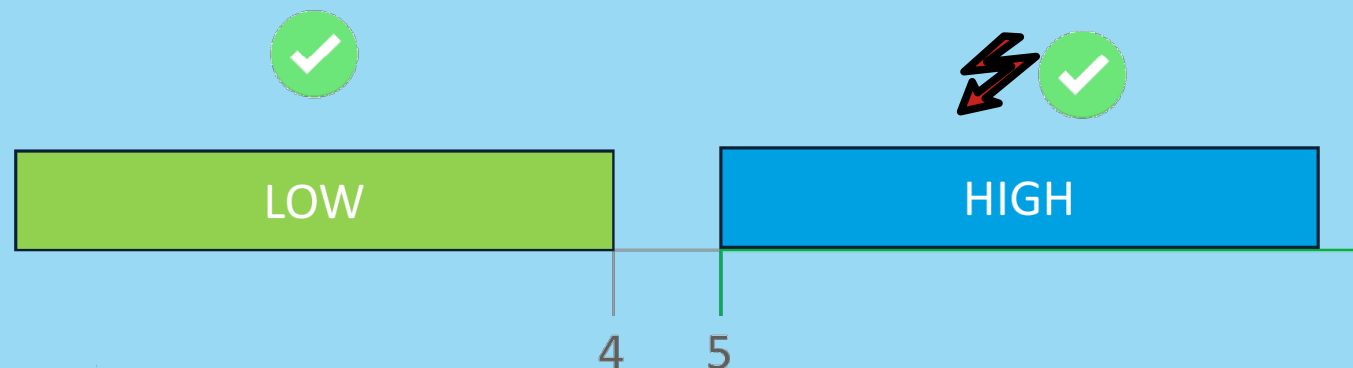


Boundary Value Analysis – Supplementary (2/4)

- Basic question: How intensively is the boundary area tested?
- In some cases it is worth adding further values close to the boundary
- Example:
 - Condition " $x \geq 5$ ": If $x \geq 5$, HIGH should be output otherwise LOW
 - Implementation:
If the code contained " $x == 5$ " instead of " $x \geq 5$ ", this error would not be found.
 - Boundary values: $x = 5$ (HIGH) and $x = 4$ (LOW)

PSEUDO-CODE

```
if (x == 5) {  
    print („HIGH“);  
} else {  
    print („LOW“);  
}
```

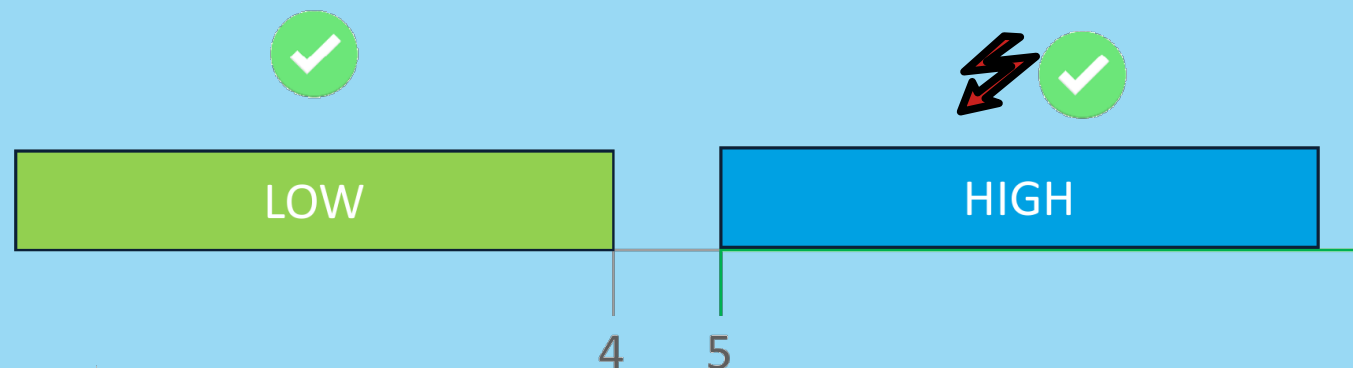


Boundary Value Analysis – Supplementary (3/4)

- Basic question: How intensively is the boundary area tested?
- In some cases it is worth adding further values close to the boundary
- Example:
 - Condition " $x \geq 5$ ": If $x \geq 5$, HIGH should be output otherwise LOW
 - Implementation:
If the code contained " $x == 5$ " instead of " $x \geq 5$ ", this error would not be found.
 - Boundary values: $x = 5$ (HIGH) and $x = 4$ (LOW)
 - Solution: Boundary value $x = 6$ would uncover the defect

PSEUDO-CODE

```
if (x == 5) {  
    print („HIGH“);  
} else {  
    print („LOW“);  
}
```

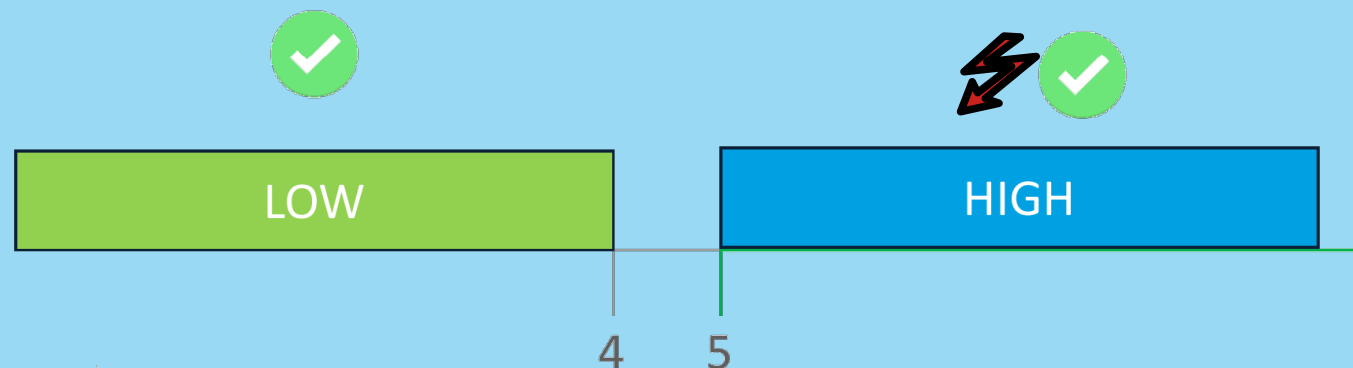


Boundary Value Analysis – Supplementary (4/4)

- Basic question: How intensively is the boundary area tested?
- In some cases it is worth adding further values close to the boundary
- Example:
 - Condition “ $x \geq 5$ ”: If $x \geq 5$, HIGH should be output otherwise LOW
 - Implementation:
If the code contained “ $x == 5$ ” instead of “ $x \geq 5$ ”, this error would not be found.
 - Boundary values: $x = 5$ (HIGH) and $x = 4$ (LOW)
 - Solution: Boundary value $x = 6$ would uncover the defect

PSEUDO-CODE

```
if (x == 5) {  
    print („HIGH”);  
} else {  
    print („LOW”);  
}
```



Adding further boundary values can make sense. It is part of risk analysis and test design.

At each boundary, test the **exact boundary value** and the **two neighboring values** (inside and outside the equivalence partition).

For floating point numbers, select a corresponding tolerance of the calculation accuracy.

Boundary Value Analysis – Hints (1/2)

- Input range boundaries
 - Range: [-1.0; +1.0]
Test data: -1.001; -1.0; +1.0; +1.001 (-0.999; +0.999)
 - Range:]-1.0;+1.0[;
Test data: -1.0; -0.999; +0.999; +1.0 (-1.001; +1.001)
- Boundaries on the permitted number of input values
 - Input file with 1 to 365 records
Test cases 0, 1, 365, 366 (2, 364) records
- Output range boundaries
 - Program calculates monetary values between EUR 0.00 and EUR 600
Test cases: For EUR 0, EUR 600 and, if possible, also for < 0; (just > 0); and for > 600; (just < 600)

Boundary Value Analysis – Hints (2/2)

- Boundaries on the permitted number of output values
 - Output of 1 to 4 points of data
Test cases: For 0, 1, 4 and 5 (2, 3) output values
- Observe first and last element for ordered sets
 - e.g. sequential file, linear list, table
- Test empty sets for complex data structures
 - e.g. empty list, zero matrix
- For numerical calculations
 - select values that are close together and far apart

Boundary Value Analysis – Advantages and Disadvantages

Advantages

- Errors are more likely to be found at the boundaries of equivalence partitions than within
- Boundary value analysis is one of the most useful methods for test case design when used correctly
- Efficient combination with other methods leave degrees of freedom in choice of test data

Disadvantages

- Difficult to specify clear-cut recipes for selection of test data
- Difficult to determine all relevant boundary values
- Creativity required to find successful test data
- Often not applied efficiently enough as it appears too simple

Chapter 4: Test Analysis and Design

Dynamic Tests: Foundations

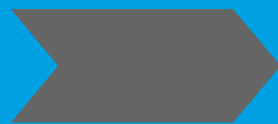
Foundations of Black-Box Testing

Equivalence Partitioning

Boundary Value Analysis

Decision Table Testing

...



Learning Objectives for Test Analysis and Design

(according to Certified Tester Foundation Level Syllabus, English Version V4.0)

- **4.1 Test Techniques Overview**
- FL-4.1.1 (K2) Distinguish black-box, white-box and experience-based test techniques
- **4.2 Black-box Test Techniques**
- FL-4.2.1 (K3) Use equivalence partitioning to derive test cases
- FL-4.2.2 (K3) Use boundary value analysis to derive test cases
- FL-4.2.3 (K3) Use decision table testing to derive test cases
- FL-4.2.4 (K3) Use state transition testing to derive test cases
- **4.3 White-box Test Techniques**
- FL-4.3.1 (K2) Explain statement testing
- FL-4.3.2 (K2) Explain branch testing
- FL-4.3.3 (K2) Explain the value of white-box testing

Decision Table Testing

- **Prerequisite:** Requirements with logical conditions and complex rules
- Procedure:
 - Examine specification
 - Determine input conditions and actions of the system
 - must be able to be either true or false

Decision Table Testing – Four Corners of Decision Table

Conditions	Rules
Actions	Action associations

Conditions

- Possible values or states of objects

Rules

- Condition indicators (combinations of condition values)

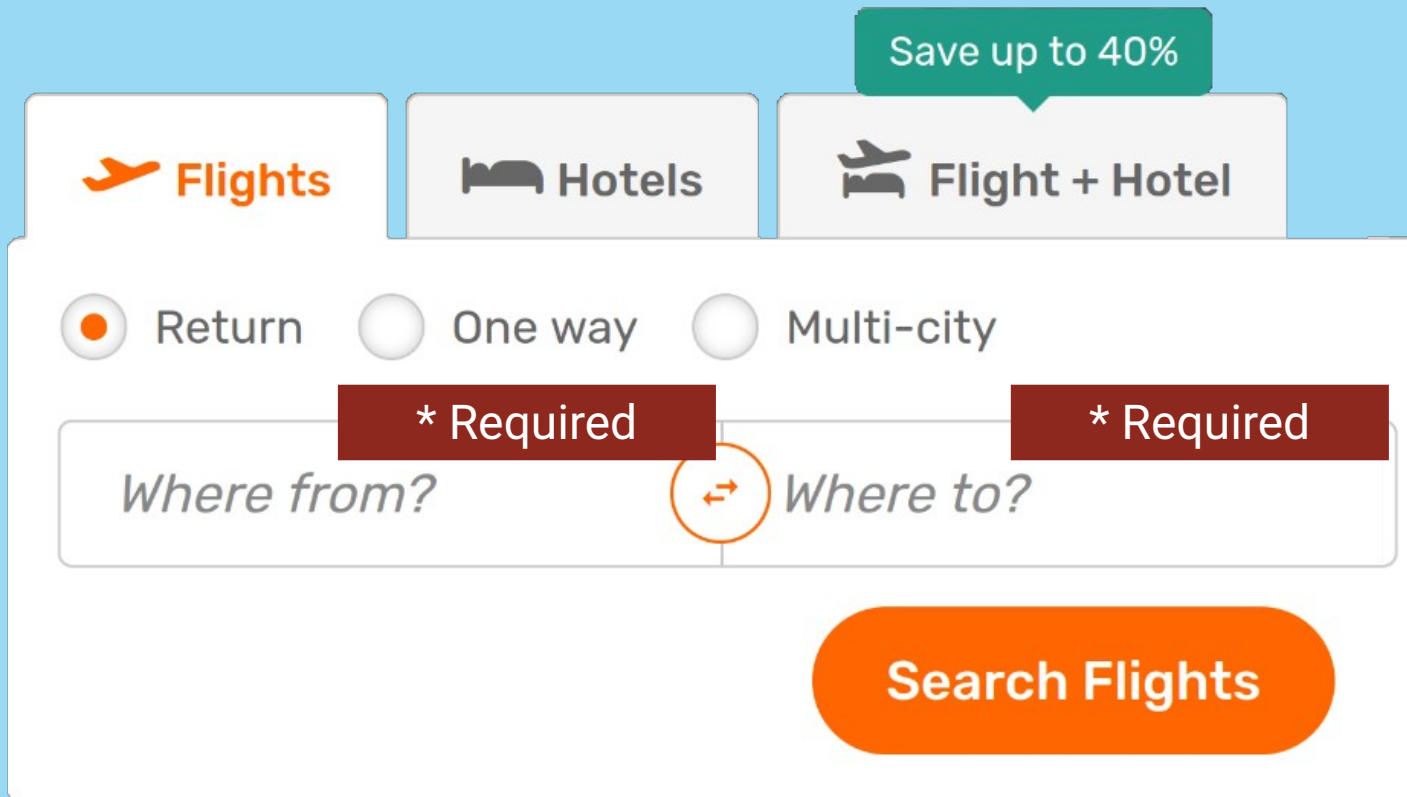
Actions

- Activities to be carried out depending on the rules

Action associations

- Assignments of actions to the conditions

Decision Table Test – First Example



The image shows a flight search form with several annotations for decision table testing. At the top, there are three tabs: 'Flights' (selected), 'Hotels', and 'Flight + Hotel'. A green callout bubble above the 'Flight + Hotel' tab says 'Save up to 40%'. Below the tabs are three radio buttons for 'Return' (selected), 'One way', and 'Multi-city'. Below these are two text input fields, 'Where from?' and 'Where to?', both marked with a red '* Required' label. A red circle with a double-headed arrow is placed between the two input fields. At the bottom is a large orange 'Search Flights' button.

- If one of the fields is empty ...
- Error notification

Decision Table Test – Example (1/8)

Conditions	Rules
Actions	Action associations

Conditions

Rules

„Where from?“ filled in?	F	T	F	T	F	T	F	T
„Where to?“ filled in?	F	F	T	T	F	T	T	T
„Return“ selected?	T	T	T	T	F	F	F	F
„One way“ selcted?	F	F	F	F	T	T	T	T

Flights

Hotels

Flight + Hotel

Save up to 40%

☒ Return
 ☐ One way
 ☐ Multi-city

* Required

Where from?

* Required

Where to?

Search Flights

Condition display:

F = not satisfied

T = satisfied

- = not relevant

N/A = undefined

Decision Table Test – Example (2/8)

Conditions	Rules
Actions	Action associations

„Where from?“ filled in?	F	T	F	T	F	T	F	T
„Where to?“ filled in?	F	F	T	T	F	T	T	T
„Return“ selected?	T	T	T	T	F	F	F	F
„One way“ selcted?	F	F	F	F	T	T	T	T
Show „Invalid Input“								
Start Return flight search with default values								
Start one way search								

Condition display:
 F = not satisfied
 T = satisfied
 - = not relevant
 N/A = undefined

Flights
 Hotels
 Flight + Hotel

☒ Return
 ☐ One way
 ☐ Multi-city

* Required

Where from?

* Required

Where to?

Decision Table Test – Example (3/8)

Conditions	Rules
Actions	Action associations

„Where from?“ filled in?	F	T	F	T	F	T	F	T
„Where to?“ filled in?	F	F	T	T	F	T	T	T
„Return“ selected?	T	T	T	T	F	F	F	F
„One way“ selcted?	F	F	F	F	T	T	T	T
Show „Invalid Input“	X							
Start Return flight search with default values								
Start one way search								

Condition display:
 F = not satisfied
 T = satisfied
 - = not relevant
 N/A = undefined

Action display:
 X = should occur
 " " = should not occur

Flights
 Hotels
 Flight + Hotel

☒ Return
 ☐ One way
 ☐ Multi-city

* Required

Where from?

* Required

Where to?

Save up to 40%

Decision Table Test – Example (4/8)

„Where from?“ filled in?	F	T	F	T	F	T	F	T
„Where to?“ filled in?	F	F	T	T	F	T	T	T
„Return“ selected?	T	T	T	T	F	F	F	F
„One way“ selcted?	F	F	F	F	T	T	T	T
Show „Invalid Input“	X	X						
Start Return flight search with default values								
Start one way search								

Condition display:
 F = not satisfied
 T = satisfied
 - = not relevant
 N/A = undefined

Action display:
 X = should occur
 " " = should not occur

Save up to 40%

Flights Hotels Flight + Hotel

☒ Return ☐ One way ☐ Multi-city

* Required * Required

Where from? ↔ Where to?

Search Flights

Decision Table Test – Example (5/8)

„Where from?“ filled in?	F	T	F	T	F	T	F	T
„Where to?“ filled in?	F	F	T	T	F	T	T	T
„Return“ selected?	T	T	T	T	F	F	F	F
„One way“ selcted?	F	F	F	F	T	T	T	T
Show „Invalid Input“	X	X	X					
Start Return flight search with default values								
Start one way search								

Condition display:
 F = not satisfied
 T = satisfied
 - = not relevant
 N/A = undefined

Action display:
 X = should occur
 " " = should not occur

The screenshot shows a flight search interface. At the top, there are three tabs: 'Flights' (with an airplane icon), 'Hotels' (with a bed icon), and 'Flight + Hotel' (with an airplane and bed icon). The 'Flight + Hotel' tab is selected, and a green badge above it says 'Save up to 40%'. Below the tabs, there are three radio buttons: 'Return' (selected with an orange dot), 'One way', and 'Multi-city'. Below the radio buttons, there are two input fields: 'Where from?' and 'Where to?'. Both fields have a red label '* Required' above them. A double-headed arrow connects the two input fields. At the bottom right, there is a large orange button that says 'Search Flights'.

Decision Table Test – Example (6/8)

„Where from?“ filled in?	F	T	F	T	F	T	F	T
„Where to?“ filled in?	F	F	T	T	F	T	T	T
„Return“ selected?	T	T	T	T	F	F	F	F
„One way“ selcted?	F	F	F	F	T	T	T	T
Show „Invalid Input“	X	X	X					
Start Return flight search with default values				X				
Start one way search								

Condition display:
 F = not satisfied
 T = satisfied
 - = not relevant
 N/A = undefined

Action display:
 X = should occur
 " " = should not occur

Flights
 Hotels
 Flight + Hotel

☒ Return
 ☐ One way
 ☐ Multi-city

* Required

Where from?

* Required

Where to?

Decision Table Test – Example (7/8)

„Where from?“ filled in?	F	T	F	T	F	T	F	T
„Where to?“ filled in?	F	F	T	T	F	T	T	T
„Return“ selected?	T	T	T	T	F	F	F	F
„One way“ selcted?	F	F	F	F	T	T	T	T
Show „Invalid Input“	X	X	X		X	X	X	
Start Return flight search with default values				X				
Start one way search								

Condition display:
 F = not satisfied
 T = satisfied
 - = not relevant
 N/A = undefined

Action display:
 X = should occur
 " " = should not occur

Save up to 40%

Flights Hotels Flight + Hotel

☒ Return ☐ One way ☐ Multi-city

* Required * Required

Where from? ↔ Where to?

Search Flights

Decision Table Test – Example (8/8)

„Where from?“ filled in?	F	T	F	T	F	T	F	T
„Where to?“ filled in?	F	F	T	T	F	T	T	T
„Return“ selected?	T	T	T	T	F	F	F	F
„One way“ selcted?	F	F	F	F	T	T	T	T
Show „Invalid Input“	X	X	X		X	X	X	
Start Return flight search with default values				X				
Start one way search								X

Condition display:

F = not satisfied

T = satisfied

- = not relevant

N/A = undefined

Action display:

X = should occur

" " = should not occur

Flights
 Hotels
 Flight + Hotel

☒ Return
 ☐ One way
 ☐ Multi-city

* Required

Where from?

* Required

Where to?

Decision Table Test

- **Prerequisite:** Requirements with logical conditions and complex rules
- Procedure:
 - Examine specification
 - Determine input conditions and actions of the system
 - must be able to be either true or false
- Each column of the table
 - defines a unique combination of conditions (rule)
 - Defines actions associated with this rule
- Commonly used standard coverage:
 - At least one test case per column

„Where from?“ filled in?	F	T	F	T	F	T	F	T
„Where to?“ filled in?	F	F	T	T	F	T	T	T
„Return“ selected?	T	T	T	T	F	F	F	F
„One way“ selcted?	F	F	F	F	T	T	T	T
Show „Invalid Input“	X	X	X		X	X	X	
Start Return flight search with default values				X				
Start one way search								X

What are advantages / disadvantages of decision table testing?

Decision Table Test – Advantages and Disadvantages

Advantages

- Combinations of conditions that might otherwise remain untested
- Decision table technique generally suitable if software processes depend on several logical decisions
- Logical relationships can be systematically formulated
- Decision tables can be easily checked for redundancy, consistency and completeness

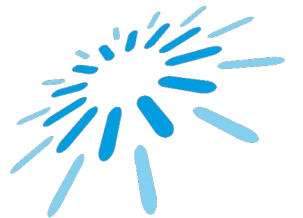
Disadvantages

- Difficult to understand if there are too many conditions
- Relationships between individual conditions can only be expressed implicitly (cf. cause-effect graph analysis)



Until next time!

Collaboration, Quality, Test
Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences